

ДЕЛОВИ КОМПАЈЛЕРА

Избор инструкција

Задатак компајлера је да програм написан у полазном језику (нпр. у Ц-у) преведе у машински код на одредишном језику, зависном од циљне архитектуре. Подела компајлера на предњи и задњи део омогућава независност полазног језика и циљне архитектуре, као и лакшу преносивост компајлера. Комуникација предњег и задњег дела компајлера подразумева увођење **међујезика** који представља:

- Циљни језик за предњи део компајлера,
- Изворни језик за задњи део компајлера.

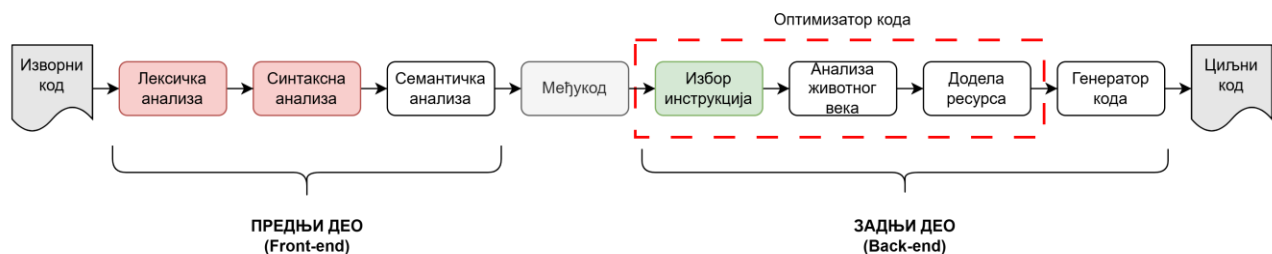
Предњи део компајлера анализира изворни програм и прави међукод, док **задњи део** на основу међукода генерише машински код за циљну архитектуру. Овај начин реализације омогућава да се исти предњи део може користити за различите архитектуре, уз промену задњег дела.



Слика 1: Предњи и задњи део компајлера (преводиоца)

$C \rightarrow \text{MIPS}$	$C \rightarrow \text{x86}$	$C \rightarrow \text{ARM}$	$C \rightarrow \text{LPRS1}$
-----------------------------	----------------------------	----------------------------	------------------------------

Фаза избора инструкција припада задњем делу компајлера и преводи међукод у код ниског нивоа који веома подсећа на машински код или асемблерски језик циљне архитектуре. У типичном компајлеру, фаза избора инструкција претходи фази заузимања стварних регистара (додела ресурса), стога излазна репрезентација ове фазе има бесконачан скуп псеудо-регистара.



Слика 2: Основне фазе компајлера (преводиоца)

Међујезик је језик који је независан од полазног језика и циљне архитектуре и представља се стаблом међукода. **Стабло међукода** садржи **шаблоне** (енгл. *Template*) који одговарају једној реалној инструкцији, (нпр. сабирање, одузимање, писање и читање из меморије) тј. описују једну основну операцију по **чвору**.

Чворови стабла могу бити:

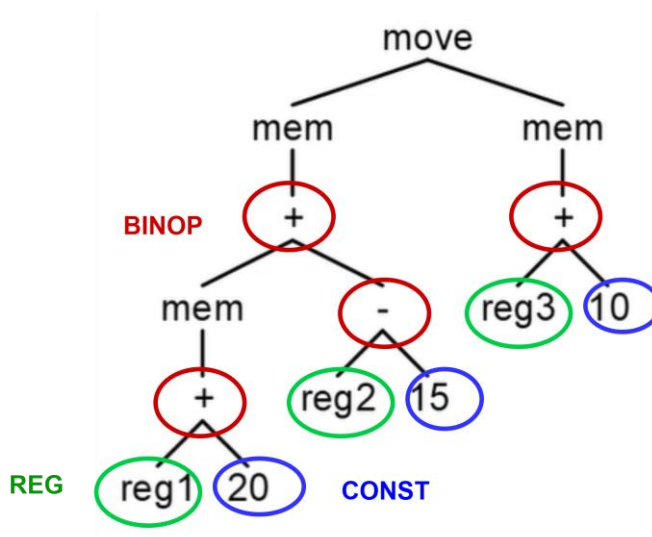
- **Изрази** – смештају резултат операције у операнд (add r3, r2, r1)
- **Искази** – не смештају резултат операције у операнд (sw r2, 20)

Програм је низ исказа где су неки искази заправо изрази. Табела 1 приказује типове исказа и изрази:

Табела 1: Типови исказа и изрази

Тип исказа	Опис
MOVE	Садржи два операнда и користи се за пребацивање садржаја из једног у други операнд.
SEQ	Користи се уколико је потребно описати два исказа у низу.
EXP	Користи се уколико исказ треба да опише само један израз.
Тип изрази	Опис
BINOP	Садржи два операнда и операцију плус, минус, итд.
ESEQ	Користи се за опис изрази који садржи по један израз и исказ.
MEM	Користи се за операцију приступа меморији.
REG	Користи се за опис изрази који садржи само један регистар.
CONST	Користи се за опис изрази који садржи само једну константу.

На Слици 3 је дат пример стабла међукода, где су чворови стабла приказани као искази и изрази. Стабло почиње са **MOVE** исказом и наставља са **MEM** изразима. Операције + и – представљају **BINOP** израз, reg1, reg2, reg3 представљају **REG** изразе, док 10, 15 и 20 представљају **CONST** изразе.



Слика 3: Пример стабла међукода са исказима и изразима

Задатак фазе избора инструкција је проналажење одговарајућег скупа инструкција којим се описује цело стабло међукода. Свака инструкција се може записати помоћу малог стабла – шаблона.

У Табели 2 су приказане основне МИПС инструкције као и одговарајући шаблони, где једна МИПС инструкција може да описује више чворова стабла међукода.

Табела 2: Основне МИПС инструкције и њихови шаблони у стаблу међуреизентације

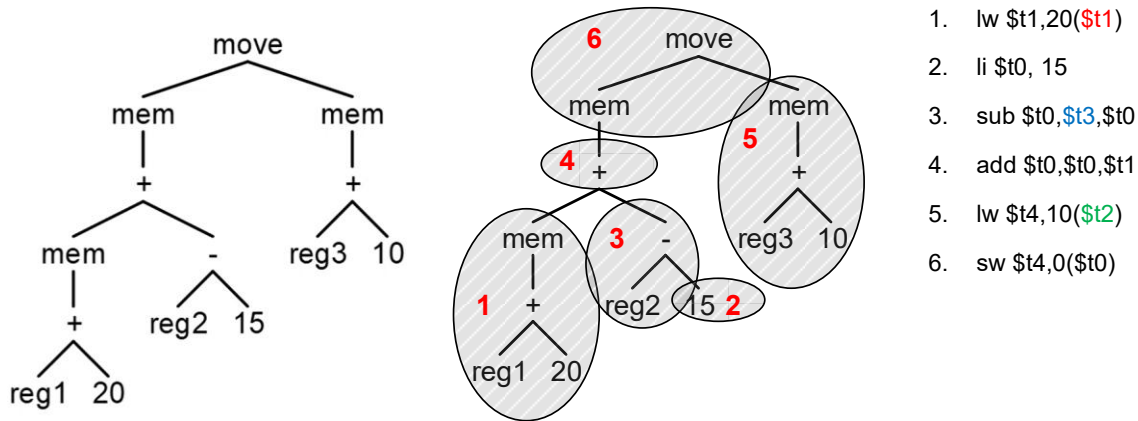
Инструкција	Пример	Шаблон	Опис
<code>add 'd, 's, 's</code>	<code>add \$r_i, \$r_j, \$r_k</code>		Сабира два операнда 's и резултат уписује у 'd.
<code>addi 'd, 's, const</code>	<code>addi \$r_i, \$r_j, 20</code>		Сабира операнд 's са константом.
<code>sub 'd, 's, 's</code>	<code>sub \$r_i, \$r_j, \$r_k</code>		Одузима два операнда 's и резултат уписује у 'd.
<code>li 'd, const</code>	<code>li \$r_i, 20</code>	<code>const</code>	Уписује константу у регистар.
<code>lw 'd, const('s)</code>	<code>lw \$r_i, 20(\$r_j)</code> <code>lw \$r_i, 20(\$zero)</code> <code>lw \$r_i, 0(\$r_j)</code>		Читање из меморије са задате адресе (базно, регистарско, са константом).
<code>sw 's, const('s)</code>	<code>sw \$r_i, 20(\$r_j)</code> <code>sw \$r_i, 20(\$zero)</code> <code>sw \$r_i, 0(\$r_j)</code>		Упис вредности у меморију на задату адресу.

За реализацију фазе избора инструкција неопходно је урадити поплочавање стабла међукода и покрити сваку инструкцију са понуђеним шаблонима.

Алгоритме за избор инструкција упоређујемо на основу цене скупа инструкција које праве. Један критеријум за одређивање цене скупа инструкција је **величина скупа**: што је скуп већи, већа је и цена. Уколико време извршавања инструкција није једнако, други критеријум би био **скуп инструкција** који се најкраће извршава. Из овога дефинишемо два појма: најповољније поплочавање и оптимално поплочавање. Под најповољнијим поплочавањем подразумева се скуп инструкција чија је цена најмања, док се под оптималним поплочавањем подразумева скуп инструкција у коме се суседне инструкције не могу спојити у нову инструкцију и тако добити скуп инструкција са нижом ценом. Један од алгоритама за избор инструкција са најповољнијим поплочавањем је Манч алгоритам.

Манч алгоритам је једноставан алгоритам за избор инструкција. Он креће од почетног чвора стабла (корена) међукода и тражи инструкцију која покрива највећи део стабла (највећи шаблон). На тај начин покриће се почетни чвор и неколико суседних чворова. Затим се алгоритам понавља за сваки преостали чвор. На основу сваког покривања стабла се праве инструкције. Манч алгоритам прави инструкције у **супротном редоследу од поплочавања**, тј. инструкције које се односе на листове стабла праве се прве, иако се последње поплочавају, јер се прво морају израчунати подизрази.

Са Сlike 4 се може видети да је корен стабла **move** чвор, али да би се направила његова инструкција (6)



Слика 4: Пример попловања стабла међукода

морају се направити друге инструкције (4, 5); и даље: да би се направила инструкција 4, морају се прво направити инструкције 1 и 3, а инструкција 3 тек после инструкције 2.

Инструкција може бити генерисана тек када су генерисане све инструкције које дефинишу њене операнде.

Као резултат попловања стабла међукода, са десне стране су приказане МИПС инструкције. Овде се може видети да је регистар 1 добио ресурс t1, регистар 2 ресурс t3 и регистар 3 ресурс t2.

Питање: Да ли у овом примеру инструкције могу бити генерисане у неком другачијем редоследу? Колико различитих могућих ваљаних редоследа генерисања инструкција постоји у овом примеру?

Реализација

Стабло међукода је описано у датотеци **Tree.h**. Стабло је описано преко исказа и израза. Разлика између њих је та да изрази крајњи резултат смештају у неки операнд, а искази не. На програм се гледа као на низ исказа, где су неки искази у ствари изрази (врста исказа EXP_STM у Табели 1). На примеру програмског језика Це можемо потврдити овај концепт, јер се може сматрати да свака наредба која се завршава са „;“ представља исказ, па би тако наредба у којој се налази само израз била управо израз који је у ствари исказ (на супрот томе треба приметити да израз може бити део исказа).

Облик стабла је структура израза (нпр. BINOP, MEM, CONST...) на основу које се бира одговарајућа MIPS инструкција:

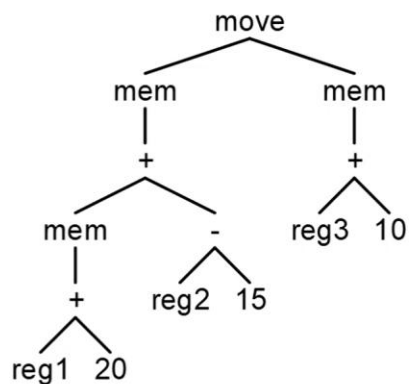
Инструкција	Шаблон (облик стабла)
add	BINOP(PLUS, EXP, EXP)
addi	BINOP(PLUS, EXP, CONST)
sub	BINOP(MINUS, EXP, EXP)
li	CONST
lw	MEM(BINOP(PLUS, CONST, REG))
sw	MOVE(MEM(...), EXP)

Искази (енгл. *statement*) су описани у класи `Statement`, док су **изрази** (енгл. *expression*) описани у класи `Expression`. За све описане подтипове израза и исказа су реализоване изведене класе (такође описане у датотеци ***Tree.h***).

Сваки тип исказа и израза поседује одговарајуће конструкторе уз помоћ којих се на једноставан начин може изградити стабло међукода. Пример за инструкцију `addi $t0,$t1,5`:

```
Expression* expr = new E_Binop(  
    E_Binop::PLUS_OP,  
    new E_Reg(reg1),  
    new E_Const(5)  
);
```

Имајући ово у виду, стабло међукода програма са Сlike 4 се може формирати на следећи начин:



```
Statement* program = new S_Move(  
    new E_Mem(new E_Binop(E_Binop::PLUS_OP,  
        new E_Mem(new E_Binop(E_Binop::PLUS_OP, new E_Const(20), new E_Reg(reg1))),  
        new E_Binop(E_Binop::MINUS_OP, new E_Const(15), new E_Reg(reg2)))),  
    new E_Mem(new E_Binop(E_Binop::PLUS_OP, new E_Const(10), new E_Reg(reg3)))  
);
```

МИПС Инструкције су описане у датотекама ***Instruction.h*** и ***Instruction.cpp***. Свака инструкција има листе одредишних и изворишних операнда (регистара) које могу бити и празне у зависности од типа инструкције (учитавање константе нема изворишне операнде). Поред тога инструкције садрже и симболички запис асемблерске инструкције у ком фигуришу одредишни и изворишни операнди у облику:

```
add `d, `s, `s
```

где се накнадно изврши замена симбола које означавају одредишне операнде „d“ (енгл. *destination*) и изворишне операнде „s“ (енгл. *source*) са стварним операндима из поменутих листа. Због тога је неопходно да листа одредишних регистара има тачно онолико елемената колико у формату инструкције има специјалних симбола „d“. Исто важи и за листу са изворишним регистрима. Класа такође располаже функцијом `toString` која добавља текстуални облик целе инструкције.

Манч алгоритам је описан у датотекама *Munch.h* и *Munch.cpp*. Ту се налазе функције `munchStm` и `munchExp` које се користе за поплочавање исказа и израза. У овим инструкцијама обавља се поплочавање међукода МИПС инструкцијама. Инструкције се праве помоћу конструктора класе **Instruction** и функције **emit** која инструкције смешта у излазну листу инструкција.

При додавању нове инструкције потребно је:

1. препознати облик стабла,
2. рекурзивно обрадити подизразе (*munchExp()*),
3. направити нови регистар ако инструкција производи резултат,
4. формирати текст инструкције,
5. попунити `src` и `dst` листе,
6. позвати *emit()*.

Следи пример за инструкцију **addi**:

```
string instructionString = "addi `d`,`s, " + toString(constExp->getValue());

Reg srcReg = munchExp(binopRightExp);
Reg dstReg = getNewReg();

Instruction* instr = new Instruction(instructionString);
instr->getDst().push_back(dstReg);
instr->getSrc().push_back(srcReg);

emit(instr);
```

Прво се формира симболички запис инструкције где се за одредишне и изворишне операнде, које тек треба одредити, поставе одговарајући симболи ``d` и ``s` који ће бити накнадно замењени. Затим се одређују изворишни и одредишни операнди. Ова инструкција има један изворишни (`srcReg`) и један одредишни (`dstReg`) операнд. Функција *getNewReg()* се користи за добављање новог јединственог регистра и користи се за одредишни операнд - операнд у који се резултат ове инструкције смешта. За изворишни операнд (цео израз) је потребно поново позвати функцију `munchExp` ради његове евалуације.

Позивом функције **emit** додаје се нова инструкција у листу инструкција `selectedInstructions`. Овој листи инструкција могуће је приступити преко следеће функције:

```
InstructionList& getInstructionList()
```

ЗАДАТАК

1. У пројекту су урађени случајеви сабирања два израза (**add**). Покрити случај сабирања са константом **addi** и одузимања два израза **sub**. Место на које треба додати инструкције је означено у коду.
2. Додати **lw** МИПС инструкцију у Манч алгоритам и тестирати је. Место на које треба додати инструкцију је означено коментаром у коду.

ДОДАТНИ ЗАДАТАК

3. Додати **sw** МИПС инструкцију у Манч алгоритам и тестирати је. Место на које треба додати инструкцију је означено коментаром у коду.